

OpenClaw终极玩法：多模型协作+轻量记忆系统完全指南

本指南将教你如何让OpenClaw发挥最大功效：多模型协同工作、成本降低70%、能力提升10倍

作者：OpenClaw创业实战营（龙虾营）
知识星球：<https://t.zsxq.com/3Pioj>
微信公众号：@开发者阿橙

核心理念

为什么多模型协作？

单一模型的问题：

- ✗ Claude擅长推理但成本高（\$15/百万token）
- ✗ GPT擅长代码但国内访问困难
- ✗ 国产模型便宜但能力有短板

多模型协作的优势：

- ✓ 成本优化：便宜的模型做重活，贵的模型只做关键决策
- ✓ 能力互补：每个模型发挥最大优势
- ✓ 降低延迟：并行处理，响应更快

最佳组合方案

模型	角色	使用场景	成本
Claude Opus 4.5	总指挥	任务规划、复杂推理、质量把控	高
GPT-5.3	代码工	代码编写、调试、重构	中
DeepSeek-V3	数据处理	大量文本分析、数据提取	低
MiniMax	内容生成	文案撰写、翻译、摘要	低
GLM-4	本地任务	简单问答、信息检索	极低

成本对比（单次复杂任务）：

单一Claude方案：~\$0.50/次
多模型协作：~\$0.15/次（节省70%）

第一步：配置多模型

1.1 准备API密钥

```
# Claude (Anthropic)
访问: https://console.anthropic.com
获取: API Key

# OpenAI (GPT)
访问: https://platform.openai.com
获取: API Key

# DeepSeek
访问: https://platform.deepseek.com
获取: API Key

# MiniMax
访问: https://api.minimax.chat
获取: API Key

# 智谱AI (GLM)
访问: https://open.bigmodel.cn
获取: API Key
```

1.2 配置OpenClaw多模型

编辑 `~/.openclaw/openclaw.json` :

```
{
  "env": {
    "ANTHROPIC_API_KEY": "sk-ant-xxxxx",
    "OPENAI_API_KEY": "sk-xxxxx",
    "DEEPSEEK_API_KEY": "sk-xxxxx",
    "MINIMAX_API_KEY": "xxxxx",
    "ZHIPU_API_KEY": "xxxxx"
  },
  "agents": {
    "defaults": {
      "model": {
        "primary": "anthropic/claude-opus-4-5",
        "fallback": [
          "openai/gpt-5.3",
          "deepseek/deepseek-v3"
        ]
      }
    }
  }
}
```

```

    ]
  }
},
"specialist": {
  "planner": {
    "model": "anthropic/claude-opus-4-5",
    "description": "任务规划、决策、质量把控"
  },
  "coder": {
    "model": "openai/gpt-5.3",
    "description": "代码编写、调试、重构"
  },
  "processor": {
    "model": "deepseek/deepseek-v3",
    "description": "大量文本处理、数据分析"
  },
  "creator": {
    "model": "minimax/text-01",
    "description": "文案撰写、内容生成"
  },
  "assistant": {
    "model": "zhipu/glm-4",
    "description": "简单问答、信息检索"
  }
}
}
}

```

1.3 创建自定义路由规则

创建 `~/.openclaw/skills/model-router.yaml` :

```

---
name: model-router
description: 智能模型路由器，根据任务类型自动选择最佳模型
version: 1.0.0

triggers:
  - type: "auto"
    description: "每次任务自动触发"

routing_rules:
  # 任务规划类 -> Claude
  - pattern:
    - "制定.*计划"

```

```
    - "设计.*方案"
    - "分析.*策略"
agent: "planner"
model: "anthropic/claude-opus-4-5"
reason: "需要深度推理和规划能力"
```

代码编写类 -> GPT

```
- pattern:
    - "写.*代码"
    - "实现.*功能"
    - "调试.*bug"
agent: "coder"
model: "openai/gpt-5.3"
reason: "代码生成能力最强"
```

大量文本处理 -> DeepSeek

```
- pattern:
    - "分析.*文档"
    - "提取.*信息"
    - "总结.*内容"
    - "处理.*数据"
agent: "processor"
model: "deepseek/deepseek-v3"
reason: "成本最低, 适合大量token处理"
```

内容创作 -> MiniMax

```
- pattern:
    - "写.*文章"
    - "生成.*文案"
    - "翻译.*内容"
agent: "creator"
model: "minimax/text-01"
reason: "中文生成质量好, 成本适中"
```

简单问答 -> GLM

```
- pattern:
    - "查询.*"
    - "搜索.*"
    - "是什么"
agent: "assistant"
model: "zhipu/glm-4"
reason: "快速响应, 成本低"
```

默认模型

```
fallback:
    model: "anthropic/claude-opus-4-5"
```

第二步：安装Swarm多Agent协作系统

2.1 快速安装

方法1：让OpenClaw自己安装

发送消息给OpenClaw：

帮我安装SwarmMemory系统，访问这个地址：
<https://github.com/rebootmindful/SwarmMemory>

安装步骤：

1. 克隆仓库到 ~/.openclaw/skills/SwarmMemory
2. 安装依赖 npm install
3. 配置 openclaw.json
4. 测试运行

方法2：手动安装

```
# 1. 克隆仓库
cd ~/.openclaw/skills
git clone https://github.com/rebootmindful/SwarmMemory.git

# 2. 安装依赖
cd SwarmMemory
npm install

# 3. 构建项目
npm run build

# 4. 测试安装
node test.js
```

2.2 集成到OpenClaw

编辑 ~/.openclaw/openclaw.json：

```
{
  "skills": {
    "enabled": [
      "@openclaw/browser",
      "@openclaw/email",
      "SwarmMemory/swarm-agent",
```

```
    "SwarmMemory/memory-system"
  ]
},
"agents": {
  "swarm": {
    "enabled": true,
    "coordinator": "planner",
    "memory": {
      "type": "vector",
      "path": "~/.openclaw/memory",
      "retention": "90d"
    },
    "handoffs": {
      "enabled": true,
      "context_transfer": "full"
    },
    "routines": {
      "auto_optimize": true,
      "learning_rate": 0.1
    }
  }
}
}
```

第三步：配置轻量记忆系统

3.1 创建记忆目录结构

```
# 创建记忆目录
mkdir -p ~/.openclaw/memory/{active,archive}

# 创建子目录
cd ~/.openclaw/memory/active
mkdir -p decisions lessons people logs tasks reflections

# 创建导航文件
touch navigation.md
```

3.2 配置记忆系统

创建 `~/.openclaw/memory/config.yaml`：

name: lightweight-memory

description: 轻量化文本记忆系统

记忆类型

memory_types:

decisions:

path: "active/decisions/"

format: "markdown"

retention: "永久"

description: "重大决策记录"

lessons:

path: "active/lessons/"

format: "markdown"

retention: "永久"

description: "可复用的经验教训"

people:

path: "active/people/"

format: "markdown"

retention: "永久"

description: "人物信息档案"

logs:

path: "active/logs/"

format: "markdown"

retention: "30d"

description: "日常日志"

tasks:

path: "active/tasks/"

format: "json"

retention: "90d"

description: "任务执行记录"

reflections:

path: "active/reflections/"

format: "markdown"

retention: "1y"

description: "周期性反思总结"

路由规则

routing_rules:

- condition: "是重大决策吗?"
destination: "decisions/"

```
    verify: true # 写入前先检查是否存在

- condition: "是可复用经验吗？"
  destination: "lessons/"
  verify: true

- condition: "是关于某人的信息吗？"
  destination: "people/"
  verify: true

- condition: "值得记录但不是知识"
  destination: "logs/{{date}}.md"
  verify: false

# 增量同步
sync:
  mode: "incremental"
  interval: "5m"
  batch_size: 100

# 遗忘机制
forgetting:
  enabled: true
  temperature_model: true
  thresholds:
    archive: 0.3
    delete: 0.1
  schedule: "weekly"

# 检索优先级
retrieval:
  level1:
    - "navigation.md" # 导航文件
  level2:
    - "decisions/"
    - "lessons/"
    - "people/"
  level3:
    type: "semantic_search"
    engine: "sqlite-vec"
    top_k: 5
```

3.3 启动记忆系统

```
# 初始化记忆系统
cd ~/.openclaw/skills/SwarmMemory
npm run initialize

# 启动记忆守护进程
npm run start:memory
```

第四步：使用多Agent协作

4.1 基础使用示例

场景1：市场推广策略制定

发送给OpenClaw：

任务：设计一个AI工具的市场推广策略

要求：

1. 目标用户：个人开发者
2. 预算：5000元/月
3. 时间：3个月

请使用Swarm多Agent协作：

- 规划Agent (Claude)：制定整体策略
- 分析Agent (DeepSeek)：分析竞品和市场
- 创意Agent (MiniMax)：撰写推广文案
- 代码Agent (GPT)：实现推广代码（如SEO工具）

执行流程：

1. [规划Agent] 接收任务 → 拆解为子任务
 - ├ 市场分析
 - ├ 创意构思
 - ├ 渠道规划
 - └ 预算评估
2. [分析Agent] 并行执行市场分析
 - 爬取竞品数据
 - 分析目标用户
 - 成本模型计算
3. [创意Agent] 撰写推广文案

- 公众号文章
- 小红书笔记
- 知乎回答

4. [代码Agent] 实现技术工具

- SEO关键词工具
- 数据统计脚本

5. [规划Agent] 汇总结果，生成最终方案

4.2 高级配置：辩论机制

创建 `~/.openclaw/skills/debate-agent.yaml`：

```
---
name: debate-agent
description: 多轮辩论与共识达成

triggers:
  - type: "keyword"
    phrases: ["辩论", "讨论", "方案对比"]

debate_config:
  rounds: 3 # 最多3轮辩论
  participants:
    - role: "创意派"
      agent: "creator"
      stance: "追求创新和差异化"

    - role: "务实派"
      agent: "planner"
      stance: "注重可行性和成本"

    - role: "市场派"
      agent: "processor"
      stance: "关注用户接受度"

  consensus_threshold: 0.7 # 70%同意即达成共识

output:
  format: "markdown"
  sections:
    - "辩论观点汇总"
    - "各方论据"
```

- "最终共识"
- "实施建议"

使用示例：

任务：为OpenClaw教程选择最佳标题

请启动辩论机制：

- 创意派：追求吸引眼球
- 务实派：注重SEO和搜索
- 市场派：关注点击率

候选标题：

1. OpenClaw完全指南
2. 10倍提升效率的AI工具
3. 个人开发者的AI助理
4. OpenClaw深度拆解

请辩论并给出最终推荐。

4.3 实时容错与重试

配置错误处理：

```
{
  "agents": {
    "error_handling": {
      "max_retries": 3,
      "retry_delay": "1s",
      "fallback_strategy": "degrade",
      "monitoring": {
        "enabled": true,
        "alert_threshold": 3
      }
    }
  }
}
```

第五步：优化与进阶

5.1 性能优化

并行执行配置：

```
parallel_execution:
  enabled: true
  max_parallel: 5 # 最多5个Agent并行

# 可并行的任务类型
parallel_tasks:
  - "数据分析"
  - "文案撰写"
  - "代码生成"

# 必须串行的任务
sequential_tasks:
  - "任务规划"
  - "最终审核"
```

成本优化技巧：

1. 批量处理：

- ✗ 逐个处理10个文档
成本： $10 \times \$0.05 = \0.50
- ✓ 批量处理10个文档
成本： $1 \times \$0.08 = \0.08

2. 智能缓存：

```
cache:
  enabled: true
  ttl: "24h"
  storage: "redis"

# 缓存相似查询
similarity_threshold: 0.85
```

3. Token预算控制：

```
{
  "budget_control": {
    "daily_limit": 100000,
    "per_task_limit": 10000,
```

```
    "alert_at": 80
  }
}
```

5.2 记忆优化

自动归档配置：

```
archival:
  schedule: "weekly"
  rules:
    - condition: "日志文件超过30天且未被引用"
      action: "archive"

    - condition: "反思文件超过1年"
      action: "archive"

    - condition: "已完成任务超过90天"
      action: "archive"

  # 永不归档
  - condition: "核心决策"
    action: "keep_forever"

  - condition: "核心经验"
    action: "keep_forever"
```

温度参数配置：

```
temperature:
  factors:
    time_decay: 0.1 # 每天降低0.1
    reference_boost: 0.5 # 被引用一次+0.5
    priority_weight: 1.0 # 优先级权重

  thresholds:
    active: 0.7 # >0.7保持活跃
    archived: 0.3 # <0.3移至归档
    delete: 0.1 # <0.1考虑删除
```

5.3 自动学习配置

```
auto_learning:
  enabled: true
```

```
# 记录成功率
success_tracking:
  - agent_combination
  - task_type
  - outcome
  - user_feedback

# 优化推荐
optimization:
  method: "bandit" # 多臂老虎机算法
  exploration_rate: 0.1
  update_frequency: "daily"

# 偏好学习
preference_learning:
  - response_style
  - detail_level
  - code_language
  - output_format
```

第六步：实战案例

案例1：自动化内容生产流水线

任务：每天自动生成并发布技术文章

```
# daily-content-pipeline.yaml
name: daily-content-pipeline

workflow:
  # 1. 热点追踪 (DeepSeek - 便宜)
  - agent: "processor"
    task: "搜索今日技术热点"
    model: "deepseek/deepseek-v3"
    output: "trends"

  # 2. 选题规划 (Claude - 重要决策)
  - agent: "planner"
    task: "基于热点选择最佳选题"
    model: "anthropic/claude-opus-4-5"
    input:
      trends: "{{steps.0.output}}"
```

```
output: "topic"
```

3. 大纲生成 (Claude - 需要深度思考)

```
- agent: "planner"
  task: "生成文章大纲"
  model: "anthropic/claude-opus-4-5"
  input:
    topic: "{{steps.1.output}}"
  output: "outline"
```

4. 内容撰写 (MiniMax - 成本低)

```
- agent: "creator"
  task: "基于大纲撰写全文"
  model: "minimax/text-01"
  input:
    outline: "{{steps.2.output}}"
    length: "3000字"
  output: "article"
```

5. 代码示例 (GPT - 代码能力)

```
- agent: "coder"
  task: "生成代码示例"
  model: "openai/gpt-5.3"
  input:
    article: "{{steps.3.output}}"
  output: "code"
```

6. 质量审核 (Claude - 关键环节)

```
- agent: "planner"
  task: "审核文章质量"
  model: "anthropic/claude-opus-4-5"
  input:
    article: "{{steps.3.output}}"
    code: "{{steps.4.output}}"
  output: "review"
```

7. 发布 (如果审核通过)

```
- condition: "{{steps.5.output.status == 'approved'}}"
  agent: "processor"
  task: "发布到多个平台"
  model: "deepseek/deepseek-v3"
  platforms:
    - "公众号"
    - "知乎"
    - "掘金"
```

```
# 成本估算
cost_estimate:
  step1: "$0.02" # DeepSeek便宜
  step2: "$0.15" # Claude贵但关键
  step3: "$0.10" # Claude
  step4: "$0.03" # MiniMax适中
  step5: "$0.05" # GPT代码
  step6: "$0.08" # Claude审核
  step7: "$0.01" # DeepSeek
  total: "$0.44" # 远低于全Claude的$0.80
```

案例2：智能客户服务系统

```
# customer-service.yaml
name: intelligent-customer-service

triggers:
  - webhook: "customer_message"
    source: "wechat"

workflow:
  # 1. 意图识别 (GLM - 快速便宜)
  - agent: "assistant"
    task: "识别客户问题类型"
    model: "zhipu/glm-4"
    categories:
      - "产品咨询"
      - "技术支持"
      - "投诉建议"
      - "其他"
    output: "intent"

  # 2. 知识检索 (向量搜索)
  - agent: "memory"
    task: "检索相关历史对话和知识库"
    search:
      type: "semantic"
      top_k: 3
    output: "context"

  # 3. 简单问题直接回答 (GLM)
  - condition: "{{steps.0.output.intent == '产品咨询'}}"
    agent: "assistant"
    task: "基于知识库回答"
    model: "zhipu/glm-4"
```

```

input:
  context: "{{steps.1.output}}"
output: "response"

# 4. 复杂问题转人工或深度处理 (Claude)
- condition: "{{steps.0.output.intent == '技术支持'}}"
  agent: "planner"
  task: "分析技术问题并给出方案"
  model: "anthropic/claude-opus-4-5"
  input:
    context: "{{steps.1.output}}"
    history: "{{steps.1.output}}"
  output: "solution"

# 5. 情感分析 (MiniMax)
- agent: "processor"
  task: "分析客户情绪"
  model: "minimax/text-01"
  input:
    message: "{{original_message}}"
  output: "sentiment"

# 6. 记录到记忆系统
- agent: "memory"
  task: "保存对话记录"
  store:
    type: "conversation"
    retention: "365d"
    tags:
      - "{{steps.0.output.intent}}"
      - "{{steps.4.output.sentiment}}"

```

第七步：持续优化

7.1 询问OpenClaw如何优化

定期问OpenClaw：

请分析我的OpenClaw使用情况：

1. 检查任务执行记录，找出：
 - 哪些Agent组合成功率最高？
 - 哪些任务类型耗时最长？

- 有没有可以优化的地方？

2. 分析记忆系统：

- 哪些知识最常被引用？
- 有没有过期需要归档的内容？
- 检索效率如何？

3. 成本分析：

- 各模型的使用比例？
- Token消耗分布？
- 有没有可以降低成本的地方？

请给出具体的优化建议。

7.2 监控指标

关键指标：

```
metrics:
  performance:
    - "任务平均响应时间"
    - "Agent切换次数"
    - "错误率"
    - "重试次数"

  cost:
    - "每日Token消耗"
    - "各模型使用比例"
    - "单任务平均成本"

  memory:
    - "检索命中率"
    - "活跃文件数量"
    - "归档文件数量"
    - "平均检索时间"

  learning:
    - "自动优化次数"
    - "偏好学习准确率"
    - "成功率提升趋势"
```

7.3 A/B测试

```
ab_testing:
  enabled: true
```

```
tests:
- name: "模型组合对比"
  variable: "agent_combination"
  variants:
    A: ["claude", "gpt"]
    B: ["claude", "deepseek", "minimax"]
  metric: "cost_per_task"
  duration: "7d"

- name: "记忆检索方式"
  variable: "retrieval_method"
  variants:
    A: "semantic_only"
    B: "hybrid"
  metric: "retrieval_latency"
  duration: "14d"
```

常见问题

Q1: 如何选择合适的模型?

决策树:

```
是否需要深度推理/规划?
├─ 是 → Claude Opus 4.5
└─ 否
    │
    │ ── 是否是代码相关?
    │   │
    │   │ ── 是 → GPT-5.3
    │   │ └─ 否
    │   │   │
    │   │   │ ── 是否是大量文本处理?
    │   │   │   │
    │   │   │   │ ── 是 → DeepSeek-V3
    │   │   │   │ └─ 否
    │   │   │   │   │
    │   │   │   │   │ ── 是否是内容创作?
    │   │   │   │   │   │
    │   │   │   │   │   │ ── 是 → MiniMax
    │   │   │   │   │   │ └─ 否 → GLM-4
```

Q2: 记忆系统会占用多少空间?

估算 (30天使用):

日志：约10MB
决策：约2MB
经验：约5MB
任务记录：约3MB
反思：约1MB
总计：约21MB（非常轻量）

Q3: 如何确保数据隐私？

建议：

1. 使用本地部署（OpenClaw默认）
2. 敏感数据不发送到云端模型
3. 定期清理记忆系统
4. 加密存储API密钥

Q4: 多Agent协作会很慢吗？

实际情况：

串行执行：Agent1 → Agent2 → Agent3

耗时：10s + 15s + 8s = 33s

并行执行：Agent1、Agent2、Agent3同时工作

耗时： $\max(10s, 15s, 8s) = 15s$

提速：54%

总结

核心要点

1. 多模型协作：
 - Claude做规划（关键决策）
 - GPT写代码（代码能力）
 - 国产模型做重活（成本优化）
2. 轻量记忆：
 - 文本化存储（便于管理）
 - 自动遗忘（保持高效）
 - 智能检索（快速定位）

3. Swarm协作：

- 动态任务分解
- 多轮辩论机制
- 自动学习优化

关键提醒

记住，一定要经常问OpenClaw：

"如何优化我的配置？"

"我的使用方式有什么可以改进的？"

"有没有更经济的模型组合？"

OpenClaw会基于你的使用情况，给出个性化建议。

资源链接

- SwarmMemory仓库：<https://github.com/rebootmindful/SwarmMemory>
- OpenClaw官网：<https://openclaw.ai>
- 记忆系统参考：<https://github.com/toby-bridges/epro-memory>

作者：OpenClaw创业实战营（龙虾营）

知识星球：<https://t.zsxq.com/3Pioj>

微信公众号：@开发者阿橙